



李文韬

☎ 19109874396 | ✉ 490542039@qq.com

📅 32 岁 | 👤 男 | 📅 6 年经验 | 🛠️ AI 应用开发工程师 | 🎓 本科 · 华南师范大学（人力资源管理） | 💰

期望薪资：面议

自我评价

4 年半 Java 后端开发 + 近一年半 AI 应用开发，项目制交付覆盖电商中台、同城社交、能源政企、AI 智能体平台等多个行业。定位 **Java 后端打底、AI 应用开发为方向**：Spring Cloud Alibaba 微服务全栈熟练，近一年半专注 Spring AI Alibaba 企业级落地，负责 RAG 混合检索（双路召回 + RRF 融合 + Rerank 重排）、Tool Calling / MCP 工具治理、会话记忆等核心链路的全流程设计与实现。

工程化 AI 落地的核心能力：不只是搭链路，而是从评估体系（RAGAS 指标 + Golden Dataset + LLM-as-Judge）、幻觉治理（检索→生成→校验→迭代四层全链路）、成本控制（缓存分层 + 模型路由）到线上监控（Token 用量归因、工具调用审计）的完整闭环。AI 编程工具深度使用者，有一套经过项目验证的 AI 协作开发方法论——将复杂需求拆解为 AI 可消化的任务单元，配合结构化 Prompt 约束，在 AskXBOT 和石化平台两个项目中累计缩短开发周期约 40%。

专业技能

一、AI 大模型应用开发

- RAG 知识库全链路**：文档解析 → 语义分块 → 双路召回（Dense + BM25）→ RRF 融合 → Rerank 重排 → 评估闭环。混合检索将召回率从 78% 提升至 92%，Rerank 后 Top5 准确率从 76% 提升至 91%；基于 RAGAS（Faithfulness / Answer Relevancy / Context Precision / Context Recall）+ Golden Dataset + LLM-as-Judge 搭建离线与在线评估体系；全链路幻觉治理（检索→生成→校验→迭代四层闭环），关键场景幻觉率从 12% 降至 2% 以下
- 智能体架构与工具治理**：路由工作流 + ReAct 循环的智能体架构设计，核心判断——简单问答不走 Agent 直出，复杂多步骤才进路由工作流（步数上限 + 超时熔断 + 重复检测）；Function Calling / MCP 工具统一适配为 ToolCallback，具备白名单授权、幂等键、执行审计与租户上下文注入的完整治理链路，工具调用正确率 96%
- 工程化成本与性能优化**：分层成本路由（缓存命中率 62% → Function Calling → RAG → LLM），单次对话 Token 成本降低 40-55%；WebFlux + SSE 流式输出，首字延迟 < 2s，用户断开后即时终止模型调用；多模型动态切换与 fallback 降级链
- Spring AI + LangChain 双栈**：Java 侧精通 Spring AI Alibaba 全组件实战（ChatClient、Advisor、

ChatMemory、VectorStore、ToolCallback)，可独立完成企业级 AI 智能体平台从 0 到 1；Python 侧熟练 LangChain + FastAPI，用于快速原型与算法实验。双栈互补而非二选一——核心链路 Java 保工程化，实验与微调 Python 保灵活性

5. AI 辅助开发方法论：不只“用 AI 写代码”——将复杂需求拆解为 AI 可消化的任务单元，配合结构化 Prompt 约束（目标 / 上下文 / 技术栈 / 输出格式 / 约束条件 / 验收标准），人工聚焦架构决策与代码审查。实际项目约 60% CRUD 与工具适配代码由 AI 辅助生成，开发周期缩短约 40%

二、Java 核心基础与底层原理

1. 熟练掌握 Java 编程及面向对象思想，精通集合框架、多线程、反射等核心特性，熟练使用 Hutool 等工具类快速实现功能
2. 深入理解 JVM 原理，包括内存模型、垃圾回收机制，掌握 JVM 调优参数及常用调优工具
3. 精通并发编程，熟悉多线程创建方式与应用场景，掌握并发安全实现原理、乐观锁与悲观锁设计思想
4. 熟悉常见数据结构，包括数组、链表、二叉树、平衡二叉树、B+ 树、跳表等，了解排序算法与二分查找

三、主流框架与后端开发

1. 熟练使用 Spring 全家桶 + MyBatis 等主流框架，深入理解 IOC、AOP、事务实现方式及原理，熟练运用模板方法模式、策略模式、工厂模式、单例模式、责任链模式等常用设计模式优化代码结构
2. 熟悉 OAuth2.0、JWT、Shiro、Sa-Token 等认证授权方案，具备统一登录、Token 鉴权与权限过滤实战经验
3. 具备网络编程基础，熟悉 HTTP、TCP、UDP、SSE、WebSocket 协议，具备 Netty 长连接服务开发经验

四、微服务与分布式架构

1. 熟悉 SpringCloud 生态，使用 Nacos 作为注册中心和配置中心、Gateway 作为网关、OpenFeign 与 Dubbo 作为远程调用工具，集成 Sentinel 实现限流熔断与 Spring Cloud LoadBalancer 负载均衡
2. 了解 Seata 分布式事务框架，理解 CAP 原理与 BASE 理论，熟悉 XA、AT 事务模式
3. 精通 Elasticsearch 分布式搜索引擎，理解倒排索引原理，可实现高亮搜索、复合查询、距离检索等功能
4. 熟练运用 RabbitMQ / RocketMQ 实现异步解耦，掌握消息消费可靠性、幂等性、有序性保障方案
5. 掌握分布式协调与调度：XXL-JOB 定时任务（配置、调度、执行、监控全链路）+ Redisson 分布式锁（可重入锁、红锁、联锁），结合 Lua 脚本实现高并发原子裁决
6. 掌握 Spring WebFlux 响应式编程，基于 Reactor 实现 SSE 流式输出与异步非阻塞处理，熟悉 Caffeine + Redis 多级缓存架构

五、数据库与缓存技术

1. 熟练使用 MySQL 数据库，精通索引、存储引擎、事务、日志等核心特性，具备 SQL 调优经验
2. 精通 Redis 缓存，熟练使用各类常用数据类型及应用场景，掌握缓存穿透、击穿、雪崩解决方案；熟悉 RDB/AOF 持久化机制，掌握主从复制搭建，理解哨兵模式与集群数据同步原理；熟练使用 Redisson 实现分布式锁与延迟队列

3. 熟悉 ClickHouse 分析型数据库，具备经营报表类聚合查询与统计场景经验
4. 熟悉 MongoDB 文档数据库，具备动态内容、消息记录等非结构化数据存储场景经验

六、运维、工具与前端基础

1. 熟悉 Linux 常用指令，熟练使用 Docker 部署项目，掌握 Jenkins + Docker 持续集成方案；熟练使用 Nginx 实现反向代理及负载均衡
2. 掌握阿里云 OSS、华为云 IOT、微信开放平台第三方授权、微信支付服务商模式等第三方服务对接
3. 熟练使用 Git、Maven、IDEA、Apifox 等开发工具；了解 Vue3 + Ant Design Vue，可开发与修改后台管理页面

工作经历

长城信息股份有限公司 | AI 应用开发工程师 / 后端开发工程师 | 2020.08 - 至今

- 2025.03 至今（近一年半 AI 应用开发）：石化经营分析平台后端小组长（带 2 人），负责 AI 助手本地化落地（LLM + RAG + Function Calling）及设备 IoT 模块；2025.07 起负责 AskXBOT 企业级 AI 智能体平台核心模块开发（Spring AI Alibaba 技术栈）
- 2020.08 - 2025.02（4 年半 Java 后端开发）：电商中台、同城社交、SaaS 零售等项目的后端核心模块开发，覆盖高并发秒杀、分布式锁、消息驱动、微信生态对接等场景

项目经历

AskXBOT 企业级 AI 智能体平台

2025.07 - 至今

技术架构：

JDK 17、Spring Boot 3.x、Spring AI Alibaba、DashScope（Qwen / DeepSeek）、Milvus、Elasticsearch、MySQL、Redis、RocketMQ、XXL-Job、Nacos、Sa-Token、WebFlux、SSE、Docker、Nginx

项目介绍：

面向企业知识问答、业务助手和流程自动化场景的 AI 智能体开发平台，提供模型管理、Bot 构建、RAG 知识库、会话记忆、Tool Calling、MCP 工具接入、应用发布和运行监控等能力。基于 Spring AI Alibaba 落地，通过 ChatClient、Advisor、ChatMemory、VectorStore 和 ToolCallback 统一模型、知识库、记忆与工具调用。

量化成果总览：

- 混合检索（Dense + BM25）召回率从单路向量的 78% 提升至 **92%**
- Rerank 重排后 Top5 准确率从 76% 提升至 **91%**
- RAGAS Faithfulness 稳定在 **0.89**，Context Precision 稳定在 **0.87**
- 高频问题缓存命中率 **62%**，分层路由后单次对话平均 Token 成本降低 **40%**
- 知识库单文档处理耗时 < 3 秒，峰值吞吐 **200 文档/分钟**

个人职责：

1、Bot 对话与会话记忆模块：

- **统一对话链路：**以 ChatClient 为统一入口，通过 Advisor 责任链组合会话记忆、RAG 检索和审计能力；根据 Bot 配置动态选择 ChatModel (Qwen / DeepSeek)，实现多模型可切换与 fallback 降级链。
- **记忆与历史分离：**通过 MessageChatMemoryAdvisor 注入最近 10 轮对话窗口（约 4K Token），完整聊天记录独立落 MySQL 支持无限回溯；对比"全量窗口注入"方案，Token 消耗降低约 55%，同时避免了长上下文导致的模型"迷失中间"问题。
- **流式输出：**基于 WebFlux + SSE 返回文本、引用来源、工具调用状态等分类事件；阻塞型调用 (JDBC、MyBatis-Plus) 切换到受控 Scheduler 避免占用 EventLoop；用户断开连接后及时终止大模型调用，避免浪费 Token；平均首字延迟控制在 **2 秒以内**。

2、RAG 知识库与混合检索模块：

- **异步入库：**文档上传后写任务表 (MySQL 作为事实源)，通过 RocketMQ 异步完成 Tika 解析、语义边界感知分块与向量化；Chunk 同时写入 Milvus 和 Elasticsearch 双索引，任务失败可重试、双索引可从事实源全量重建。
- **双路召回与重排：**查询时向量检索 (Milvus) 与 BM25 (ES) 并行召回 Top50 → 按 Chunk ID 去重 → RRF 融合排序 → BGE-Reranker 交叉编码精排 Top5 → 注入 Prompt。单路超时自动降级到另一路，不放大故障。
- **评估体系：**构建 200 条覆盖制度查询、业务操作、故障排查等典型场景的 Golden Dataset，基于 RAGAS 框架定期跑回归评估 (Faithfulness、Answer Relevancy、Context Precision、Context Recall 四项指标)，新版本上线前对比基线。Rerank 引入后 Context Precision 从 68% 提升至 87%，Answer Relevancy 从 72% 提升至 85%。
- **幻觉治理 (全链路)：**检索侧靠混合检索 + Rerank 保证上下文精准；生成侧 Prompt 强约束——"若检索文档无相关信息，直接回答'当前知识库未收录该内容'，禁止编造"；校验侧对输出做事实一致性检查 + 引用溯源；迭代侧建立 Bad Case 登记→根因分析→对照实验→数据验证的闭环机制，关键场景事实性幻觉率从 12% 降至 2% 以下。
- **权限前置：**知识权限 (ACL) 进入 Milvus Metadata Filter 与 ES Filter 检索条件，注入 Prompt 前再做一次业务校验，双保险防越权引用。

3、Tool Calling 与 MCP 工具接入模块：

- **工具统一治理：**将内部 Java 方法 (@Tool 注解)、动态 HTTP 接口和外部 MCP 工具统一适配为 ToolCallback，按 Bot 建立工具白名单；通过 MCP Client Starter 接入标准 MCP Server。
- **安全与审计：**ToolContext 由服务端注入租户与用户上下文 (不信任模型生成的身份参数)；写操作工具要求幂等键与二次确认；执行日志记录 traceId、toolName、耗时与结果，支持按 Bot + 时间范围的审计回溯。
- **Bad Case 驱动优化：**早期版本模型调用工具时频繁传错参数名 (如将 deviceId 写成 device_id)，根因是工具描述与模型理解之间存在 gap——通过优化 @Tool 的 description (加入参数示例 + 边界说明) 配合 Few-Shot Prompt，工具调用正确率从 82% 提升至 96%。

技术架构：

Spring Boot、Spring Cloud Alibaba、Nacos、Gateway、OpenFeign、Sentinel、RabbitMQ、Redis、XXL-Job、MyBatis-Plus、MySQL、ClickHouse、Spring AI、Milvus、Git、Apifox

项目介绍：

面向能源企业省公司、地市公司和油站一线人员的经营分析与业务管理平台，覆盖设备管理、经营台账、非油商品、客户核算、报表分析、告警通知和 AI 智能问答等场景。平台服务约 **14 个地市、近 2,000 座油站**，目标是统一经营数据口径、提升设备异常处理效率，并降低一线人员查询制度和操作咨询的成本。

量化成果：AI 助手上线后，一线人员制度与操作查询平均耗时从 **8 分钟降低至 30 秒以内**；设备离线告警从发现到推送平均延迟 < 30 秒；缓存 + 分层路由策略使单次问答平均 Token 成本降低 **55%**。

个人职责：

1、设备 IoT 接入与状态监控模块：

- **设备数据接入：**设备上报数据进入 RabbitMQ 后异步消费，避免上报高峰直接打满业务接口；消费端刷新 Redis TTL 心跳 Key，结合 XXL-Job 定时扫描判断设备离线状态。
- **离线告警闭环：**对未恢复告警做状态维护，避免每次扫描重复生成告警；设备恢复上报后更新设备状态并恢复告警，形成"离线发现 → 告警推送 → 恢复消除"的完整闭环。

2、AI 助手本地化落地模块：

- **分层成本路由（核心亮点）：**高频问题优先走 Redis 语义缓存（命中率约 58%，延迟 < 50ms）；能查业务数据的问题走 Function Calling（模型选工具 → 后端校验参数 → 查数据库，延迟 ~1.5s）；无法通过工具解决的问题再进入 RAG 检索 + 大模型生成（延迟 ~3s）。通过缓存 + 路由分流，单次问答平均 Token 成本降至不分层的 45%。
- **Function Calling 工具封装：**将设备查询、告警查询、客户概览等 Java 业务能力封装为 Spring AI Tool，模型只负责选择工具和生成参数，后端负责参数校验、权限过滤和真实数据查询——"模型做决策，后端做执行"，工具调用正确率约 94%。
- **RAG 知识库接入：**基于 Milvus 实现制度文档的向量检索，结合 Reranker 重排序提升召回质量，回答附带引用来源；检索相关性不足（相似度 < 0.75）时走兜底话术，不强行生成——这条规则源于早期 Bad Case（模型在无关文档上强行总结出错误操作流程，导致一线人员误操作投诉）。
- **流式交互：**基于 SSE 实现回答流式输出，结合会话历史管理支持多轮对话。

3、后端小组长与跨模块联调：

- 任后端小组长，带 2 人，负责团队内部分工和关键模块落地；配合非油、客户核算、网关通知等模块联调，按请求链路、Token 权限、参数绑定、SQL、缓存、日志逐层定位并解决问题。

启山智软 SmartShop 电商中台

2022.06 - 2023.05

技术架构：

JDK 11、Spring Boot 2.7+、Spring Cloud Alibaba、Dubbo 3、Nacos、Spring Cloud Gateway、Sentinel、MyBatis-Plus、MySQL、Redis、Redisson、RabbitMQ、Elasticsearch、XXL-Job、OSS、Docker

项目介绍：

经过百万真实用户检验的电商业中台系统，通过订单中心、会员中心、营销中心、商家中心、商品中心、财务中心等模块沉淀多种商业模式的共性能力，支持 S2B2C、B2B2C、O2O、B2C 等多种商城形态。本人负责营销中心下的秒杀活动与优惠券两个模块。

个人职责：

1、秒杀活动模块：

- **分层过滤逐级削峰：** 前端置灰与验证码 → Nginx IP 限流 → Gateway Sentinel 限流 → Redis Lua 原子裁决 → MQ 异步下单，数据库只承担与库存量同级的真实订单量，系统承载能力从约 500 QPS 提升至 8,000+ QPS。
- **Lua 原子裁决：** 库存预热到 Redis，Lua 脚本原子完成"库存校验 → 一人一单校验 → 预扣减 → 记录已购用户"，关闭并发窗口；裁决失败直接返回错误码，不进入后续流程。
- **异步下单与补偿闭环：** 预扣成功发送 RabbitMQ 消息异步创建订单，消费失败重试耗尽进入死信队列，补偿任务回滚 Redis 库存与已购标记，杜绝"库存扣了订单没有"的悬挂状态。

2、优惠券系统模块：

- **券资产全生命周期：** 券模板与用户券分离，支持满减、折扣、无门槛等类型和手动领取、兑换码、定向发放等方式，覆盖创建、发放、领取、使用、核销、返还、过期完整状态机。
- **并发安全分层选型：** 超发用乐观锁条件更新原子裁决 (`issue_num < total_num`)；用户限领属于"查 → 判断 → 插入"新增场景，用 Redisson 分布式锁（粒度 `userId + couponId`）。
- **锁与事务边界排查（经典 Bad Case）：** 定位并修复"锁释放早于事务提交"导致的限领突破问题——通过 Spring Ordered 调整锁切面优先级（@Lock 切面 `order = 0`，@Transactional 切面 `order = 1`，保证先拿锁再开事务、先提交再释放），并用 `AopContext.currentProxy()` 代理对象调用解决自调用事务失效。此问题排查过程形成了完整的并发安全排查方法论。

邻圈同城社交

2021.07 - 2022.05

技术架构：

Spring Cloud Alibaba、SpringBoot、MyBatis-Plus、MySQL、Redis、MongoDB、RabbitMQ、Elasticsearch、Netty、WebSocket、XXL-Job、Nacos、Gateway、OSS

项目介绍：

基于地理位置的同城社交应用，主打"发现身边有趣的人和事"，支持图文/视频动态、兴趣圈子、线下活动报名、实时聊天和位置共享等功能，分为用户端（App/小程序）和管理端（内容审核、用户管理、数据统计）。用户规模约 5 万 DAU。

个人职责：

1、附近的人模块：

- 基于 Redis GEO 实现附近用户搜索与距离排序，位置 30 分钟未更新自动过期；热门城市按城市拆分 GEO Key 分片（北京/上海/广州各一个 Key），配合更新防抖（5 分钟内重复上报跳过）与 Caffeine 本地缓存扛住高频位置上报；支持隐藏位置、模糊距离等隐私保护策略。

2、内容审核与信用体系模块：

- 采用"先发后审"策略提升用户体验：内容发布后立即展示，同时发 MQ 异步调用第三方审核 API，违规内容自动下架并通知用户；配套用户信用分体系，按违规程度阶梯处罚（限发、禁言、封号），良好行为可恢复分数。

店堂主（WEIIT）数字零售 SaaS 系统

2020.08 - 2021.07

技术架构：

Spring Cloud、OAuth2.0、Shiro、JWT、Gateway、Nacos、MyBatis、MySQL（Schema 隔离）、Redis、Nginx、Ant Design Vue

项目介绍：

面向中小企业的数字化零售 SaaS 系统，帮助企业一键生成独立品牌的小程序商城、公众号商城和 PC 商城，涵盖商品、订单、会员、营销、自定义装修、微信生态对接和支付等核心模块，支持多租户数据隔离。

个人职责：

1、营销中心模块：

- 实现拼团（Redis 记录参团人数、XXL-Job 扫描过期未成团自动退款）、优惠券（乐观锁防超发 + Redis 计数防重复领取）、积分（规则配置化、流水记录、余额缓存）等营销工具。

2、微信生态对接模块：

- 基于微信开放平台第三方平台能力，实现小程序一键授权、ext.json 配置注入、模板代码自动上传与批量升级；接管公众号菜单、素材与群发；token 加密存储并自动刷新。